МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Школа разработки видеоигр

Практическое занятие №2

«Реализация ECS-архитектуры и визуализация объектов»

по дисциплине «Архитектура игровых движков»

Выполнили:

студенты группы J3300

Власов Д. Н.

Шабанов Т. О.

Проверил:

доцент практики

Павлович А. А.

г. Санкт-Петербург

2026 год

СОДЕРЖАНИЕ

ВВЕДЕНИЕ....3
1. Цель работы.... 3
2. Задачи работы.... 3
ХОД РАБОТЫ....4
1. Описание и архитектура.... 4
2. Технологический стек.... 4
3. Реализация ECS–ядра (Entity, Component, World/Registry).... 4
4. Базовые компоненты (Transform, Tag, MeshRenderer).... 5
5. Система рендеринга.... 5
6. Интеграция ECS в главный цикл приложения.... 6
7. Работа игрового движка.... 6
ЗАКЛЮЧЕНИЕ....8
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ....9
ПРИЛОЖЕНИЕ 1
ПРИЛОЖЕНИЕ 2
ПРИЛОЖЕНИЕ 3

ВВЕДЕНИЕ

1. Цель работы

Целью данного практического занятия является разработка и интеграция в существующий каркас игрового движка системы Entity-Component-System (ECS), обеспечивающей композиционное построение игровых объектов и их визуализацию через RenderAdapter, созданный в предыдущей работе.

2. Задачи работы

Задачами данного практического занятия являются:

1) Реализация базовых классов и структур для Entity-Component-System.

2) Создание реестра (World/Registry) для управления сущностями и компонентами.

3) Реализация компонентов: Transform (позиция, поворот, масштаб), Tag (имя объекта), MeshRenderer (тип примитива, цвет/текстура, материал).

4) Создание системы рендеринга, которая использует RenderAdapter для отрисовки всех сущностей с компонентами Transform и MeshRenderer.

5) Интеграция ECS в игровой цикл приложения.

6) Обеспечение отрисовки нескольких объектов с разными трансформациями.

7) Реализация поддержки иерархии объектов (Hierarchy) с преобразованием локальных координат в мировые.

ХОД РАБОТЫ

1. Описание и архитектура

В ходе практического занятия базовая архитектура игрового движка NeneEngine была расширена путём добавления слоя ECS, который позволяет динамически конструировать игровые объекты из независимых компонентов и обрабатывать их через специализированные системы.

Ссылка на репозиторий с исходным кодом игрового движка NeneEngine на платформе хостинга репозиториев GitHub представлена в Приложении 1.

2. Технологический стек

Для работы с векторами, кватернионами и матрицами технологический стек игрового движка NeneEngine был расширен математической библиотекой glm.

3. Система сборки

Игровой движок NeneEngine использует двухуровневую систему сборки на базе CMake.

Полный листинг корневого CMake-файла CMakeLists.txt продемонстрирован в Приложении 2.

Полный листинг CMake-файла CMakeLists.txt в папке src продемонстрирован в Приложении 3.

4. Реализация ECS–ядра (Entity, Component, World/Registry)

Были определены базовые элементы ECS-архитектуры. Сущность Entity представляет собой уникальный идентификатор и реализована как тип entt::entity. Компоненты являются простыми структурами данных без поведения. Центральным классом стал World, который содержит экземпляр entt::registry и предоставляет шаблонные методы для создания сущностей, добавления, получения и удаления компонентов. Кроме того, в World

реализована регистрация систем и их последовательный вызов в методах Update и Render.

Ключевой листинг класса World:

```
class World {
public:
    Entity CreateEntity(const std::string& name = "Entity");
    template<typename Component, typename... Args>
    Component& AddComponent(Entity entity, Args&&... args);
private:
    entt::registry m_registry;
    std::vector<std::unique_ptr<ISystem>> m_systems;
};
```

5. Базовые компоненты (Transform, Tag, MeshRenderer)

Реализованы три основных компонента согласно заданию. Компонент Transform содержит позицию, кватернион поворота и масштаб объекта, а также метод GetModelMatrix() для построения матрицы модели. Компонент Tag хранит строковое имя сущности и автоматически добавляется при создании объекта. Компонент MeshRenderer определяет тип примитива, флаг видимости, идентификатор меша и материал с цветом tint.

Ключевой листинг компонента Transform:

```
glm::mat4 Transform::GetModelMatrix() const {
    glm::mat4 model = glm::mat4(1.0f);
    model = glm::translate(model, position);
    model = model * glm::mat4_cast(rotation);
    model = glm::scale(model, scale);
    return model;
}
```

6. Система рендеринга

Разработана система RenderSystem, наследующая интерфейс ISystem. В методе Render система находит активную камеру, вычисляет матрицы view и projection, после чего получает все сущности с компонентами Transform и MeshRenderer. Для каждой видимой сущности формируется структура RenderItem, содержащая модельно-видеопроекционную матрицу и параметры примитива, и передаётся в существующий RenderAdapter. Таким образом, отрисовка полностью интегрирована с предыдущей реализацией DiligentDX12Adapter.

Ключевой листинг класса RenderSystem:

```
for (auto entity : view) {
    const auto& transform = view.get<Transform>(entity);
    const auto& meshRenderer = view.get<MeshRenderer>(entity);
    if (!meshRenderer.visible) continue;
    RenderItem item{};
    item.modelViewProjectionMatrix = viewProjectionMatrix *
transform.GetModelMatrix();
    item.primitiveType = meshRenderer.primitiveType;
    item.tint = meshRenderer.material.tint;
    m_renderer->SubmitRenderItem(item);
}
```

7. Интеграция ECS в главный цикл приложения

Интеграция выполнена в классе NeneEngineApp. В методе Init создаётся экземпляр World, в который добавляются все системы (CameraControllerSystem, MovementSystem и RenderSystem), после чего загружается тестовая сцена. В главном цикле Run после обработки ввода и обновления состояний вызываются m_world.Update(deltaTime) и m_world.Render(). Это обеспечило автоматическую обработку всех компонентов и систем каждый кадр без изменения существующей логики приложения.

Ключевой листинг класса NeneEngineApp:

```
m_world.AddSystem(std::make_unique<ECS::RenderSystem>(m_renderer.get()));
TestScene::LoadOrCreate(m_world, width, height);
// ...
m_world.Update(deltaTime);
m_renderer->BeginFrame();
m_world.Render();
m_renderer->EndFrame();
```

8. Работа игрового движка

При запуске приложения в окне отображается четыре примитива с разными трансформациями, цветами и поведением (один объект перемещается по сцене). Камера управляет клавиатурой и мышью. Все объекты корректно рендерятся через существующий DiligentDX12Adapter.

Демонстрация работы игрового движка представлена на рисунке 1.

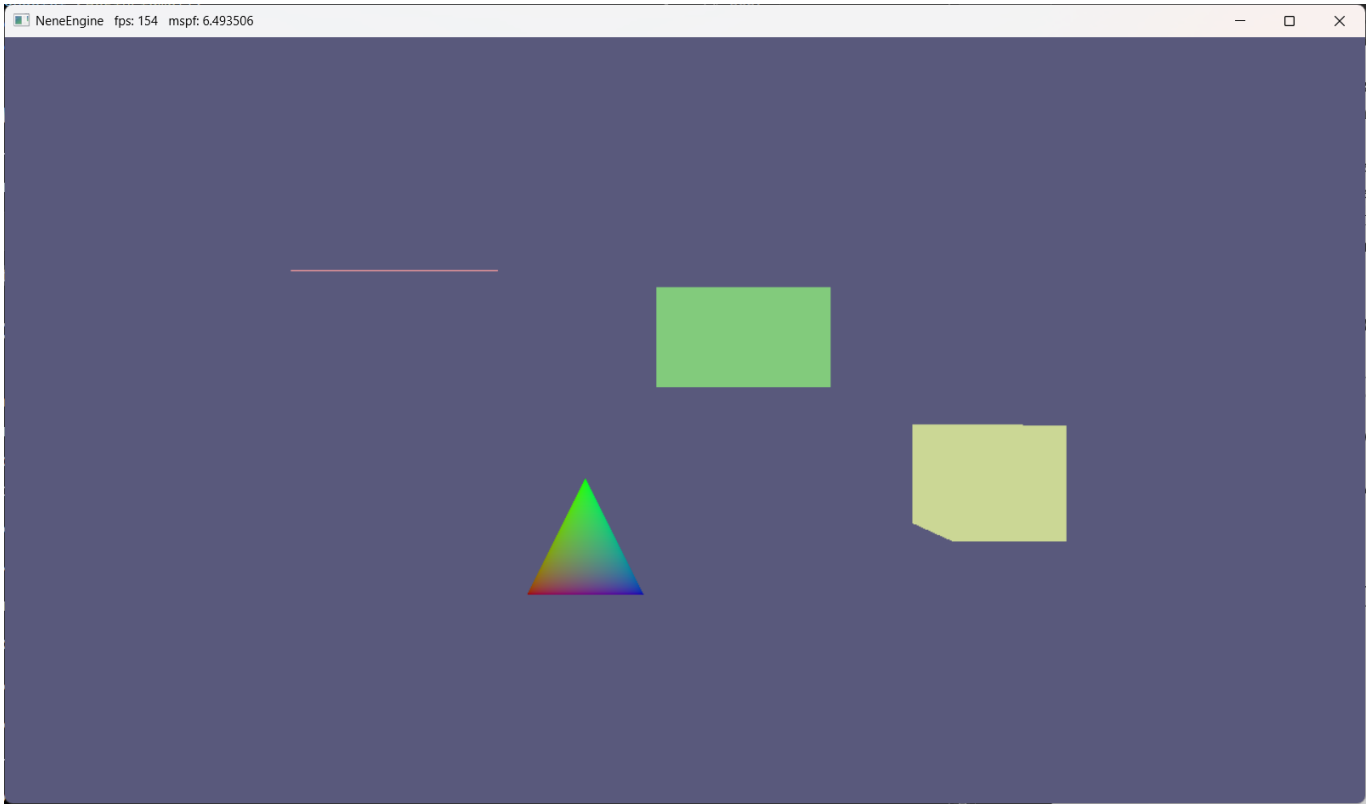

Рис. 1 Отрисовка тестовых геометрических примитивов в демонстрационном окне

ЗАКЛЮЧЕНИЕ

В ходе выполнения данного практического занятия была разработана и интегрирована система Entity-Component-System, что позволило построить игровые объекты композиционно и обеспечить их визуализацию. Реализация соответствует поставленным задачам и использует библиотеку EnTT для эффективного управления компонентами. Полученный каркас готов к дальнейшему расширению новыми системами и компонентами.

Работа выполнена в полном объёме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Джейсон Г. Игровой движок. – СПб.: Издательский дом Питер, 2021 г. – 1136 с.

2. Нильстрём Р. Паттерны программирования игр. – М.: Эксмо, 2021 г. – 432 с.

3. Купер Дж. Основы архитектуры игровых движков. – М.: Наука, 2018 г. – 2022 с.

4. https://github.com/electronicarts/EASTL/tree/master/doc – документация альтернативной стандартной библиотеки шаблонов EASTL.

5. https://spdlog.readthedocs.io/en/latest/ – документация библиотеки логирования spdlog.

6. https://diligentgraphics.github.io/docs/ – документация графического фреймворка Diligent Graphics.

7. https://learn.microsoft.com/en-us/windows/win32/api/ – документация API оконной системы Windows Win32API.

8. https://nlohmann.github.io/json/ – документация библиотеки для работы с JSON-форматом nlohmann/json.

9. https://skypjack.github.io/entt/ – документация библиотеки Entity Component System EnTT.

ПРИЛОЖЕНИЕ 1

Репозиторий игрового движка NeneEngine на платформе GitHub

https://github.com/BennySpace/NeneEngine

ПРИЛОЖЕНИЕ 2

Листинг корневого CMake-файла CMakeLists.txt

```
# NeneEngine/CMakeList.txt : Top-level CMake project file, do global
configuration
cmake_minimum_required (VERSION 3.12)
project (
             NeneEngine
             VERSION 0.1
             DESCRIPTION "Game engine"
             LANGUAGES CXX
)
set(CMAKE_CXX_STANDARD          20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS        OFF)
# Diligent Engine configs
set(DILIGENT_BUILD_SAMPLES OFF CACHE BOOL "" FORCE)
set(DILIGENT_BUILD_TOOLS   OFF CACHE BOOL "" FORCE)
set(DILIGENT_BUILD_FX      OFF CACHE BOOL "" FORCE)
set(DILIGENT_BUILD_TESTS   OFF CACHE BOOL "" FORCE)
set(DILIGENT_NO_DIRECT3D11 ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_VULKAN     ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_OPENGL     ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_GLES       ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_METAL      ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_DIRECT3D12 OFF CACHE BOOL "" FORCE)
add_subdirectory ("external/DiligentEngine/DiligentCore")
foreach(target_name SPIRV-Tools-opt SPIRV-Tools SPIRV-Tools-core)
    if(TARGET ${target_name})
        get_target_property(aliased_target ${target_name} ALIASED_TARGET)
        if(aliased_target)
            target_compile_options(${aliased_target} PRIVATE /wd5232 /wd4717)
            set_target_properties(${aliased_target} PROPERTIES
COMPILE_WARNING_AS_ERROR OFF)
            message(STATUS "[Diligent Fix] Applied to aliased target:
${aliased_target}")
        else()
            target_compile_options(${target_name} PRIVATE /wd5232 /wd4717)
            set_target_properties(${target_name} PROPERTIES
COMPILE_WARNING_AS_ERROR OFF)
            message(STATUS "[Diligent Fix] Applied to target:
${target_name}")
        endif()
    endif()
endforeach()
add_subdirectory ("src")
# add_subdirectory(tests)
```

ПРИЛОЖЕНИЕ 3

Листинг CMake-файла CMakeLists.txt для CPP-файлов

```
# NeneEngine/src/CMakeList.txt : CMake project for NeneEngine, include source
and define project specific logic here.
file(GLOB_RECURSE SOURCES CONFIGURE_DEPENDS
    "${CMAKE_CURRENT_SOURCE_DIR}/*.cpp"
             "${CMAKE_CURRENT_SOURCE_DIR}/include/*.h"
    "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp"
)
list(FILTER SOURCES EXCLUDE REGEX
    "(.*_test\\.cpp$|.*Test\\.cpp$|.*\\.t\\.cpp$|.*main_test\\.cpp$)"
)
add_executable(NeneEngine WIN32 ${SOURCES} "${CMAKE_SOURCE_DIR}/main.cpp")
target_include_directories(NeneEngine
             PUBLIC

             $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/../include>

             $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/../external>
)
find_package(spdlog                                  CONFIG REQUIRED)
find_package(nlohmann_json             CONFIG REQUIRED)
find_package(EASTL                                   CONFIG REQUIRED)
find_package(EnTT                                    CONFIG REQUIRED)
target_link_libraries(NeneEngine
             PRIVATE
                         spdlog::spdlog
                         nlohmann_json::nlohmann_json
                         EASTL
                         EnTT::EnTT
                         Diligent-GraphicsEngineD3D12-shared
)
if(COMMAND copy_required_dlls)
    copy_required_dlls(NeneEngine)
else()
    message(WARNING "copy_required_dlls не найдена — возможно, Diligent не
подгрузился")
endif()
set_property(DIRECTORY ${CMAKE_CURRENT_SOURCE_DIR} PROPERTY
VS_STARTUP_PROJECT NeneEngine)
target_compile_features(NeneEngine PUBLIC cxx_std_20)
```